

Deteccion de GPS Falso (Fake GPS)

Seguridad de la Informacion - Android / Flutter

1. Descripcion

Se implemento un mecanismo de deteccion de aplicaciones de ubicacion simulada (Fake GPS) en la actividad principal de la aplicacion. Al iniciar, la app consulta al sistema operativo si alguna app tiene el permiso de mock location activo. En caso positivo, bloquea el acceso mostrando una pantalla de error en lugar del login.

2. Que es Fake GPS?

Fake GPS es una tecnica donde una aplicacion falsifica la ubicacion GPS del dispositivo. Android permite esto mediante la funcion "Ubicacion de prueba simulada" en las Opciones de Desarrollador, pensada originalmente para pruebas de desarrollo. Las apps maliciosas abusan de esta funcion para engañar a otras aplicaciones sobre la ubicacion real del usuario.

3. Archivos modificados

MainActivity.kt	Deteccion nativa: usa AppOpsManager para verificar si alguna app tiene mock location activo
mock_location_service.dart	Servicio Flutter: llama al codigo nativo mediante MethodChannel
login_page.dart	Vista de login: verifica al iniciar y bloquea la pantalla si detecta Fake GPS

4. Fragmento 1 - MainActivity.kt (deteccion nativa Android)

```
package com.ameth.seguridad

import android.app.AppOpsManager
import android.content.Context
import android.content.pm.PackageManager
import android.os.Build
import android.os.Bundle
import android.provider.Settings
import android.view.WindowManager
import io.flutter.embedding.android.FlutterActivity
import io.flutter.embedding.engine.FlutterEngine
import io.flutter.plugin.common.MethodChannel

class MainActivity : FlutterActivity() {
    private val CHANNEL = "com.ameth.seguridad/security"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        window.setFlags(
            WindowManager.LayoutParams.FLAG_SECURE,
            WindowManager.LayoutParams.FLAG_SECURE
        )
    }
}
```

```

override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
    super.configureFlutterEngine(flutterEngine)
    MethodChannel(flutterEngine.dartExecutor.binaryMessenger, CHANNEL)
        .setMethodCallHandler { call, result ->
            when (call.method) {
                "isFakeGpsInstalled" -> result.success(isFakeGpsInstalled())
                else -> result.notImplemented()
            }
        }
}

private fun isFakeGpsInstalled(): Boolean {
    // API < 23: ajuste de sistema (metodo antiguo)
    @Suppress("DEPRECATION")
    if (Build.VERSION.SDK_INT < 23) {
        if (Settings.Secure.getInt(contentResolver,
            Settings.Secure.ALLOW MOCK_LOCATION, 0) != 0) return true
    }

    // API 23+: verificar con AppOpsManager
    val appOps = getSystemService(Context.APP_OPS_SERVICE) as AppOpsManager
    val packages = packageManager.getInstalledPackages(0)

    for (pkg in packages) {
        if (pkg.packageName == packageName) continue
        val uid = pkg.applicationInfo?.uid ?: continue
        val mode = if (Build.VERSION.SDK_INT >= 29) {
            appOps.unsafeCheckOpNoThrow(
                AppOpsManager.OPSTR MOCK_LOCATION, uid, pkg.packageName)
        } else {
            @Suppress("DEPRECATION")
            appOps.checkOpNoThrow(
                AppOpsManager.OPSTR MOCK_LOCATION, uid, pkg.packageName)
        }
        if (mode == AppOpsManager.MODE_ALLOWED) return true
    }
    return false
}
}

```

5. Fragmento 2 - mock_location_service.dart (servicio Flutter)

```

import 'package:flutter/services.dart';

class MockLocationService {
    static const _channel = MethodChannel('com.ameth.seguridad/security');

    static Future<bool> isFakeGpsInstalled() async {
        try {
            return await _channel.invokeMethod<bool>('isFakeGpsInstalled') ?? false;
        } on PlatformException {
            return false;
        }
    }
}

```

6. Fragmento 3 - login_page.dart (verificacion al iniciar)

```
bool _fakeGpsDetected = false;

@override
void initState() {
  super.initState();
  _checkFakeGps();
}

Future<void> _checkFakeGps() async {
  final detected = await MockLocationService.isFakeGpsInstalled();
  if (mounted) setState(() => _fakeGpsDetected = detected);
}

@override
Widget build(BuildContext context) {
  if (_fakeGpsDetected) return _FakeGpsBlockScreen();
  return Scaffold( /* login normal */ );
}
```

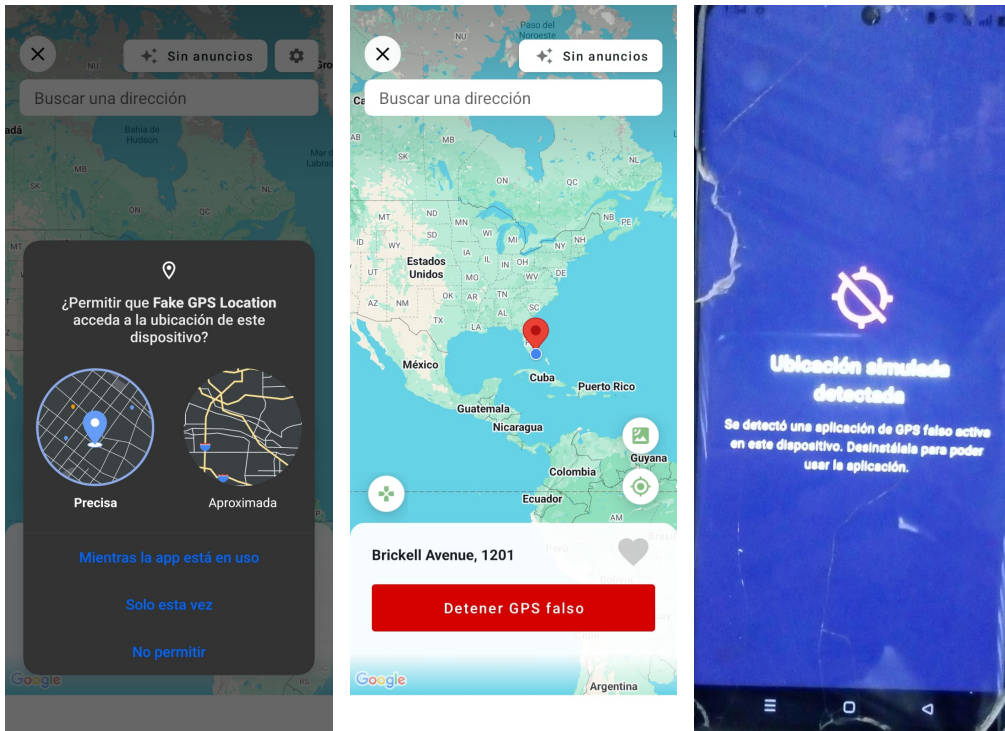
7. Explicacion tecnica

MethodChannel	Canal de comunicacion bidireccional entre Flutter (Dart) y el codigo nativo Android (Kotlin)
AppOpsManager	Servicio del SO Android que gestiona permisos de operaciones sensibles en tiempo real
OPSTR MOCK_LOCATION	Identificador de la operacion de ubicacion simulada dentro de AppOpsManager
MODE_ALLOWED	Constante que indica que el permiso esta activo para esa aplicacion
isFakeGpsInstalled()	Itera todas las apps instaladas y verifica si alguna tiene mock location autorizado
initState()	Se ejecuta una sola vez al crear el widget; ideal para verificaciones de seguridad iniciales

8. Flujo de ejecucion

1. La app inicia -> LoginPage ejecuta initState()
2. initState() llama a MockLocationService.isFakeGpsInstalled() en segundo plano
3. Flutter envia la llamada al canal "com.ameth.seguridad/security"
4. MainActivity.kt recibe la llamada y ejecuta isFakeGpsInstalled()
5. AppOpsManager revisa todas las apps instaladas buscando MODE_ALLOWED
- 6a. Si detecta Fake GPS -> se muestra pantalla de bloqueo con mensaje de error
- 6b. Si no detecta nada -> la app muestra el login normalmente

9. Prueba realizada (Capturas de pantalla)



Instalamos una aplicación de fake gps para cambiar la ubicación del dispositivo, se dieron permisos para que la aplicación pudiera realizar cambios en el sistema operativo, después en la aplicación detecta correctamente que se esta usando una aplicación para cambiar la ubicación del dispositivo y bloquea el acceso

10. Prueba realizada

App de prueba	Fake GPS Location (Lexa) - disponible en Google Play Store
Configuracion	Opciones de Desarrollador -> Seleccionar app de ubicacion simulada -> Fake GPS
Con Fake GPS activo	La app muestra pantalla de bloqueo con icono y mensaje de error
Sin Fake GPS	La app muestra el login normalmente
Capturas bloqueadas	FLAG_SECURE impide tomar capturas (pantalla negra al intentarlo)

Nota: AppOpsManager.OPSTR MOCK_LOCATION requiere API 23 (Android 6.0) o superior. Para versiones anteriores se utiliza Settings.Secure.ALLOW MOCK_LOCATION.